

MULTI-STRATEGY TRADING BACKTESTER

Reusable Plugin Architecture, Implementation Roadmap & Agile Sprint Plan

Designed for a generic backtesting engine where strategies are replaceable plugins

Version 1.0 | September 2026

Executive Summary

The project should be designed as a general-purpose backtesting framework with pluggable strategies, not as a backtester hard-wired to one trading strategy. The engine handles market data, execution, risk, portfolio accounting, analytics, persistence, and reporting. Individual strategies only decide when and how they want to trade.

The intended user experience is simple: launch the program, view all available strategies, choose one, select the instrument and historical period, and run only that strategy. A future strategy should be addable without rebuilding the data engine, broker simulator, portfolio system, analytics, database, or reporting layer.

```
python main.py
```

```
=====
BACKTESTING ENGINE
=====
```

Available strategies:

```
[1] ICT MSS + BISI/SIBI + OTE
[2] EMA Crossover
[3] RSI Mean Reversion
[4] Breakout Strategy
[5] Moving Average Trend
```

Select strategy: 1

Selected:

ICT MSS + BISI/SIBI + OTE

Instrument:

```
[1] EURUSD
[2] GBPUSD
[3] XAUUSD
```

Select: 1

Time period:

2023-01-01 -> 2025-12-31

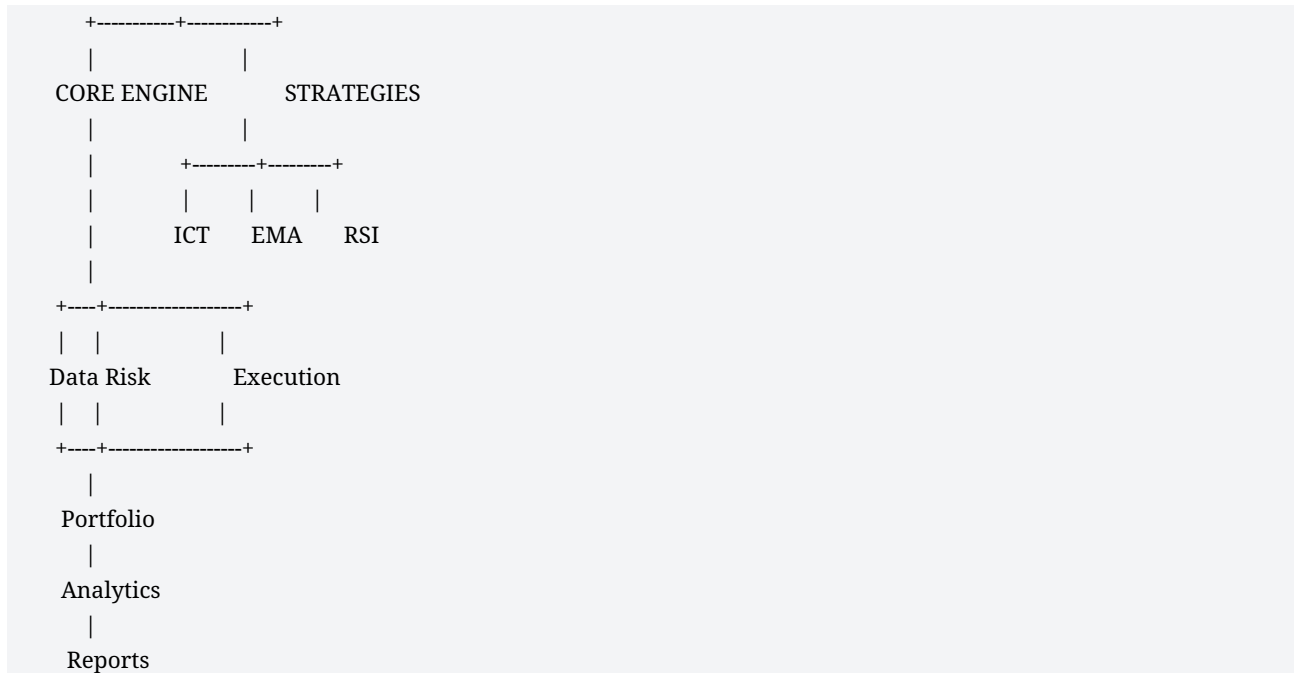
Starting backtest...

1. Fundamental Architecture

Think of the framework like a game console: the backtesting framework is the console and strategies are the games. You do not rebuild the console each time you add a new game. The same principle should govern the codebase.

```
BACKTESTING PLATFORM
```

```
|
```



A strategy is not allowed to become the backtester. It only communicates decisions to the engine: what it sees, whether it wants to enter, the direction, entry conditions, invalidation/stop conditions, targets, and cancellation conditions.

2. The Strategy Contract

Every strategy must speak the same language to the engine. Internally, strategies can be completely different, but externally they implement a common interface. This is the core abstraction that makes the system reusable.

```

class BaseStrategy:

    initialize()

    on_candle()

    generate_signal()

    reset()

    get_metadata()
  
```

The engine does not need to understand ICT, RSI, EMA, breakout logic, or any other strategy. It only needs to call the strategy interface and receive either no signal or a standardized TradeSignal.

3. Recommended Project Architecture

```

trading_backtester/
|
|-- README.md
|-- pyproject.toml
  
```

```

|-- .env
|-- .gitignore
|-- main.py
|
|-- app/
|   |-- __init__.py
|   |-- cli.py
|   |-- menu.py
|   `-- runner.py
|
|-- config/
|   |-- __init__.py
|   |-- settings.py
|   |-- instruments.py
|   |-- data.py
|   `-- backtest.py
|
|-- core/
|   |-- __init__.py
|   |-- enums.py
|   |-- models.py
|   |-- events.py
|   |-- exceptions.py
|   `-- constants.py
|
|-- data_engine/
|   |-- __init__.py
|   |-- downloader.py
|   |-- loader.py
|   |-- cleaner.py
|   |-- validator.py
|   |-- resampler.py
|   |-- cache.py
|   `-- repository.py
|
|-- indicators/
|   |-- __init__.py
|   |-- common/
|   |   |-- atr.py
|   |   |-- ema.py
|   |   |-- sma.py
|   |   |-- rsi.py
|   |   `-- volatility.py
|   `-- market_structure/
|       |-- swings.py
|       |-- mss.py
|       |-- fvg.py
|       |-- bisi_sibi.py
|       |-- volume_imbalance.py
|       `-- ote.py
|
|-- strategies/

```

```

| |-- __init__.py
| |-- base/
| | |-- __init__.py
| | |-- strategy.py
| | |-- context.py
| | |-- signal.py
| | `-- metadata.py
| |-- registry.py
| |-- discovery.py
| |-- ict/
| | |-- __init__.py
| | |-- strategy.py
| | |-- config.py
| | |-- state.py
| | `-- README.md
| |-- ema_crossover/
| | |-- __init__.py
| | |-- strategy.py
| | |-- config.py
| | `-- README.md
| |-- rsi_reversion/
| | |-- __init__.py
| | |-- strategy.py
| | |-- config.py
| | `-- README.md
| `-- breakout/
|   |-- __init__.py
|   |-- strategy.py
|   |-- config.py
|   `-- README.md
|
|-- engine/
| |-- __init__.py
| |-- backtester.py
| |-- event_loop.py
| |-- broker.py
| |-- execution.py
| |-- order_manager.py
| `-- position_manager.py
|
|-- risk/
| |-- __init__.py
| |-- models.py
| |-- sizing.py
| |-- limits.py
| `-- manager.py
|
|-- portfolio/
| |-- __init__.py
| |-- portfolio.py
| |-- balance.py
| |-- equity.py

```

```
|  `-- positions.py
|
|-- analytics/
|   |-- __init__.py
|   |-- metrics.py
|   |-- performance.py
|   |-- drawdown.py
|   |-- expectancy.py
|   `-- comparison.py
|
|-- reporting/
|   |-- __init__.py
|   |-- report.py
|   |-- csv_report.py
|   |-- html_report.py
|   `-- json_report.py
|
|-- visualization/
|   |-- __init__.py
|   |-- candles.py
|   |-- trades.py
|   |-- equity.py
|   `-- dashboard.py
|
|-- database/
|   |-- __init__.py
|   |-- connection.py
|   |-- models.py
|   |-- repositories.py
|   `-- migrations/
|
|-- plugins/
|   |-- __init__.py
|   |-- loader.py
|   `-- validator.py
|
|-- optimization/
|   |-- __init__.py
|   |-- parameter_grid.py
|   |-- runner.py
|   `-- parallel.py
|
|-- utils/
|   |-- __init__.py
|   |-- logger.py
|   |-- time.py
|   |-- ids.py
|   `-- profiler.py
|
|-- scripts/
|   |-- download_data.py
|   |-- validate_data.py
```

```

| |-- list_strategies.py
| |-- run_backtest.py
| `-- generate_report.py
|
|-- tests/
| |-- unit/
| | |-- core/
| | |-- indicators/
| | |-- strategies/
| | |-- execution/
| | `-- risk/
| |-- integration/
| |-- regression/
| `-- fixtures/
|
|-- data/
| |-- raw/
| |-- cleaned/
| |-- processed/
| |-- cache/
| `-- results/
|
|-- logs/
|
`-- documentation/
    |-- architecture.md
    |-- strategy_api.md
    |-- adding_a_strategy.md
    |-- data_pipeline.md
    |-- engine.md
    |-- testing.md
    `-- development.md

```

The key separation is between the generic framework and strategy packages. The framework owns infrastructure; strategy packages own strategy-specific behavior and state.

4. BaseStrategy Interface

```

from abc import ABC, abstractmethod

class BaseStrategy(ABC):

    @property
    @abstractmethod
    def name(self) -> str:
        pass

    @abstractmethod
    def initialize(self, context):
        pass

    @abstractmethod

```

```

def on_candle(self, context):
    pass

@abstractmethod
def generate_signal(self, context):
    pass

@abstractmethod
def reset(self):
    pass

```

A concrete strategy that fails to implement the required methods should be rejected before a backtest begins. This creates a stable contract and prevents strategy-specific code from leaking into the engine.

5. StrategyContext

Do not pass many unrelated DataFrames and objects into every strategy method. Wrap the information a strategy is allowed to access inside a StrategyContext. This improves readability, makes dependency boundaries explicit, and helps reduce accidental look-ahead bias.

```

@dataclass
class StrategyContext:
    timestamp: datetime
    market: MarketData
    indicators: IndicatorStore
    portfolio: PortfolioSnapshot
    instrument: Instrument
    timeframe: Timeframe
    config: StrategyConfig

```

6. Universal TradeSignal Model

```

@dataclass
class TradeSignal:
    strategy_id: str
    timestamp: datetime
    symbol: str
    direction: Direction
    entry_type: OrderType
    entry_price: float | None
    stop_loss: float | None
    take_profit: float | None
    confidence: float | None
    metadata: dict

```

An ICT strategy and an EMA strategy can return completely different metadata while still using the same universal signal structure. The broker and risk engine consume the standardized fields and do not care why the signal was created.

7. Strategy Metadata and Requirements

```
class ICTStrategy(BaseStrategy):
    name = "ICT MSS + BISI/SIBI + OTE"
    strategy_id = "ict_mss_ote"
    version = "1.0.0"
    required_timeframes = ["1m", "15m"]
    required_indicators = ["swings", "mss", "bisi_sibi", "ote"]
```

The strategy should declare what it needs. The framework can then prepare only the required timeframes and indicators instead of computing everything for every run.

8. Strategy-Specific Configuration

Avoid a single global settings file containing parameters for every possible strategy. Each strategy package should own its own configuration. This keeps the system modular and prevents configuration from becoming an unmaintainable collection of unrelated constants.

```
# strategies/ict/config.py
class ICTConfig:
    swing_strength = 3
    ote_threshold = 0.60
    session_start = ...
    session_end = ...
    max_stop = ...

# strategies/ema_crossover/config.py
class EMAConfig:
    fast_period = 20
    slow_period = 50

# strategies/rsi_reversion/config.py
class RSISConfig:
    period = 14
    oversold = 30
    overbought = 70
```

9. Strategy Discovery and Registry

There are three useful maturity levels for strategy discovery.

Level 1 - Manual Registry

```
STRATEGIES = {
    "ict": ICTStrategy,
    "ema": EMACrossoverStrategy,
    "rsi": RSISStrategy,
}
```

This is easy to implement, but requires editing registry.py whenever a new strategy is added.

Level 2 - Automatic Discovery (Recommended)

The framework scans the strategies directory, imports each strategy module, verifies that it contains a valid BaseStrategy subclass, reads metadata, and makes the strategy available automatically. This satisfies the long-term goal that adding a strategy package makes it appear in the menu without changing the core engine.

```
strategies/  
|-- ict/  
|-- ema_crossover/  
|-- rsi_reversion/  
`-- breakout/
```

Level 3 - Installed Plugin Architecture

At a later stage, strategies can become independent Python packages discovered through package metadata/entry points. This would allow separately installed strategies to appear in the framework. It is powerful, but unnecessary for the first version. Build Level 2 first.

10. Thin main.py and Application Layer

```
from app.runner import Application  
  
def main():  
    app = Application()  
    app.run()  
  
if __name__ == "__main__":  
    main()
```

main.py should only bootstrap the application. Strategy discovery, CLI interaction, configuration loading, engine construction, and report generation should live in dedicated modules.

```
Startup  
|  
Discover strategies  
|  
Display strategies  
|  
User selects one  
|  
Load strategy configuration  
|  
Select instrument  
|  
Select historical range  
|  
Prepare required data  
|  
Create BacktestEngine  
|  
Inject selected Strategy
```

```
|  
Execute  
|  
Analyze  
|  
Report
```

11. Dependency Injection

The backtester must never create a specific strategy internally. Instead, the selected strategy is injected into the engine. This prevents the engine from depending on one concrete strategy.

```
# Avoid  
class Backtester:  
    def __init__(self):  
        self.strategy = ICTStrategy()  
  
# Prefer  
class Backtester:  
    def __init__(self, strategy):  
        self.strategy = strategy  
  
strategy = ICTStrategy()  
engine = Backtester(strategy)
```

12. Responsibilities of the BacktestEngine

The engine should know about market data, the current timestamp, orders, positions, balance/equity, execution, risk rules, and the selected strategy. It should not contain knowledge about BISI, OTE, RSI, EMA crosses, or any other strategy-specific concept.

13. Shared Indicators vs Strategy Logic

Reusable mathematical indicators should be implemented once and shared by any strategy that needs them. ATR, EMA, SMA, RSI and volatility measures belong in common indicators. Market-structure primitives such as swings, MSS, FVG, BISI/SIBI and OTE can live in a reusable market-structure indicator package.

```
IndicatorRegistry  
|-- ATR  
|-- EMA  
|-- RSI  
|-- SwingDetector  
|-- MSSDetector  
|-- FVGDetector  
`-- OTE
```

14. Strategy State and Isolation

Complex strategies are stateful. For example, an ICT implementation might move through a sequence of waiting for a swing, waiting for MSS, defining a range, identifying a BISI/SIBI, waiting for retracement, validating OTE, and finally generating an entry. That state belongs inside the ICT strategy package, not inside BacktestEngine.

```
WAITING_FOR_SWING
|
WAITING_FOR_MSS
|
MSS_CONFIRMED
|
RANGE_FORMED
|
BISI_IDENTIFIED
|
WAITING_FOR_RETRACE
|
OTE_REACHED
|
WAITING_FOR_CONFIRMATION
|
ENTRY
```

Each new backtest must call `strategy.reset()` so no state, indicator value, pending setup, or signal survives from a previous run.

15. Backtest Configuration vs Strategy Configuration

```
# Backtest configuration
instrument: EURUSD
start_date: 2024-01-01
end_date: 2025-01-01
initial_balance: 5000
spread_model: fixed
strategy: ict_mss_ote

# Strategy configuration
swing_strength: 3
ote_threshold: 0.60
mss_timeframe: 15m
execution_timeframe: 1m
```

16. Instrument Abstraction

```
Instrument(
    symbol="EURUSD",
    asset_class="forex",
    pip_size=0.0001,
```

```
    decimals=5,  
)
```

Instrument conventions should be centralized so strategies do not hard-code pip sizes, decimal precision, or symbol-specific assumptions.

17. Strategy-Declared Data Requirements

```
required_timeframes = {"15m", "1m"}
```

If another strategy requires 4H, 1H and 15M, the DataEngine should prepare those timeframes for that run. If a strategy needs only 1M, the framework should avoid unnecessary resampling.

18. Event-Driven Backtest Architecture

A clean long-term design is event-driven. New candles create candle events. Strategies react and may produce signals. Risk converts valid signals into orders. The broker simulator processes orders into fills. The portfolio consumes fills and tracks positions, cash, balance and equity.

```
New Candle  
|  
CANDLE_EVENT  
|  
Strategy  
|  
SIGNAL_EVENT  
|  
Risk Manager  
|  
ORDER_EVENT  
|  
Broker Simulator  
|  
FILL_EVENT  
|  
Portfolio
```

19. Broker, Order and Trade Models

The broker simulator should know only generic order concepts such as buy/sell, market/limit/stop, quantity, entry, stop-loss and take-profit. It should never contain strategy-specific checks such as if bisi or if ema > ema50.

```
Order(  
    id=...,  
    strategy_id=...,  
    symbol=...,  
    side=BUY,  
    order_type=MARKET,  
    quantity=...,  
    entry_price=...,
```

```

    stop_loss=...,
    take_profit=...
)
Trade(
    trade_id=...,
    backtest_id=...,
    strategy_id=...,
    instrument=...,
    direction=...,
    entry_time=...,
    entry_price=...,
    exit_time=...,
    exit_price=...,
    stop_loss=...,
    take_profit=...,
    pnl=...,
    r_multiple=...,
    metadata=...
)

```

The metadata field allows each strategy to store research context without changing the universal trade schema. ICT metadata can hold swing/MSS/BISI/OTE information; EMA metadata can hold fast/slow EMA values; RSI metadata can hold RSI values.

20. StrategyRegistry Responsibilities

- Discover strategies
- Validate strategy implementations
- Store strategy metadata
- List available strategies
- Instantiate the selected strategy
- Reject invalid or duplicate plugins

```

registry.list()
# -> ict_mss_ote, ema_crossover, rsi_reversion, breakout

registry.create("ict_mss_ote")
# -> ICTStrategy instance

```

21. CLI and Execution Modes

Interactive Mode

```

$ backtester backtest

Select Strategy:
> ICT MSS + OTE

Select Instrument:
> EURUSD

```

```
Start Date:
> 2023-01-01

End Date:
> 2025-12-31

Initial Balance:
> 5000
```

Non-Interactive Mode

Automation and batch experiments should not depend on menus. The CLI should eventually support command-line arguments.

```
backtester run \
  --strategy ict_mss_ote \
  --symbol EURUSD \
  --start 2023-01-01 \
  --end 2025-12-31
```

Batch Backtesting

Once non-interactive execution exists, the same engine can run multiple strategies, instruments, date windows and parameter sets automatically. This enables fair strategy comparison on identical historical data.

22. Technology Stack

Technology	Purpose
Python	Entire framework
NumPy	Numerical calculations
pandas	Historical/time-series processing
PyArrow	Parquet storage
Pydantic	Configuration/data validation
pydantic-settings	Typed application settings
pytest	Automated testing
pytest-cov	Coverage measurement
Ruff	Linting and formatting checks
mypy	Static type checking
httpx	Historical-data APIs
Matplotlib	Basic plots
Plotly	Interactive charts
PostgreSQL	Persistent backtest/results storage
SQLAlchemy	Database abstraction/ORM
Psycopg	PostgreSQL driver
Alembic	Database migrations

Numba	Performance optimization
joblib	Parallel backtest/parameter runs
Streamlit	Optional dashboard
tqdm	Progress bars
Git	Version control
GitHub	Remote repository/issues

The plugin design itself does not require a third-party plugin framework. Python standard-library tools such as `abc`, `importlib`, `inspect`, `dataclasses`, `typing`, `enum` and `pathlib` are sufficient for the first version.

23. Software-Engineering Concepts Introduced

- Abstract Base Classes / interfaces
- Dependency Injection
- Registry Pattern
- Plugin Architecture
- Factory Pattern
- Event-Driven Architecture
- Domain Models
- Strategy Pattern
- Repository Pattern
- Configuration Management
- State Machines
- Dependency Inversion

The central idea is the Strategy Pattern: `BacktestEngine` depends on `BaseStrategy`, not on `ICTStrategy` or any other concrete implementation.

24. Testing Strategy

Framework Tests

- `DataEngine`
- `Execution/Broker`
- `Risk`
- `Portfolio`
- `Analytics`
- `Strategy Registry`
- `Plugin Loader`

Indicator Tests

- `ATR`
- `EMA`
- `RSI`
- `Swing detection`
- `MSS`

- FVG
- BISI/SIBI
- OTE

Strategy Tests

Every strategy gets its own unit and regression tests. Adding Strategy #10 should not require rewriting the framework tests.

Contract Tests

- Inherits BaseStrategy
- Unique strategy ID
- Provides human-readable name
- Declares required timeframes
- initialize() succeeds
- reset() succeeds
- generate_signal() returns TradeSignal or None
- Metadata is valid

25. Logging and Reproducibility

```
[BACKTEST: BT-0021]
[STRATEGY: ict_mss_ote]
[EURUSD]
LONG opened
```

Every log line associated with a run should include the backtest ID and strategy ID. This makes debugging multiple strategies and simultaneous experiments much easier.

26. Database Model

```
strategies
|
v
backtest_runs
|
+-----+
v       v
trades   equity_points
|
v
signals
```

strategies

```
strategy_id
name
version
description
created_at
```

backtest_runs

```
backtest_id
strategy_id
strategy_version
strategy_config
instrument
start_date
end_date
starting_balance
created_at
git_commit
```

trades

```
trade_id
backtest_id
strategy_id
entry
exit
direction
SL
TP
PnL
R
metadata
```

Strategy versioning and Git commit tracking make results reproducible. A result should always be traceable to the exact strategy version, configuration and code revision that generated it.

27. Phase-Wise Agile Roadmap

Phase	Estimate	Deliverable / Goal
Phase 0 - Requirements & Specification	4 h	Define strategy contract, data access, signal schema, engine/strategy/risk/broker responsibilities. Deliver architecture.md and strategy_api.md.
Phase 1 - Foundation	5 h	Project structure, venv, dependencies, logging, config, pytest, Ruff, Git. Goal: python main.py boots cleanly.
Phase 2 - Core Domain Models	6 h	Candle, Instrument, Signal, Order, Fill, Position, Trade, PortfolioSnapshot, enums and exceptions.
Phase 3 - Strategy API	6 h	BaseStrategy, StrategyContext, TradeSignal, StrategyMetadata, StrategyConfig. Build a DummyStrategy to test architecture.
Phase 4 - Strategy Registry	5 h	register, list, find, create and validate strategies.
Phase 5 - Automatic Discovery	6 h	Scan strategies/ and discover valid implementations without editing the engine.

Phase 6 - CLI/Application Layer	5 h	Strategy selection, instrument/date selection, configuration loading and backtest launch.
Phase 7 - Data Engine	10 h	Download, load, clean, validate, normalize, cache and resample historical data.
Phase 8 - Indicator Framework	6 h	Indicator interface, registry and caching, then reusable indicator modules.
Phase 9 - Execution Engine	10 h	Order, Fill, Broker, Position and execution simulation. Run DummyStrategy end-to-end.
Phase 10 - Risk Engine	6 h	Position sizing, stop validation, portfolio limits and risk rules.
Phase 11 - Portfolio	5 h	Cash, balance, equity, open/closed positions and PnL.
Phase 12 - Event/Backtest Engine	10 h	CandleEvent, SignalEvent, OrderEvent, FillEvent and the complete event loop.
Phase 13 - Analytics	7 h	Win/loss rates, expectancy, average R, profit factor, drawdown, streaks and equity curve.
Phase 14 - First Real Strategy: ICT	18 h	Swings, MSS, BISI/SIBI, FVG, OTE and strategy state machine inside the generic framework.
Phase 15 - Second Strategy	4 h	Implement a simple EMA crossover as the architectural proof that the engine is genuinely reusable.
Phase 16 - Database	7 h	Persist strategies, runs, signals, orders, trades, equity and metrics.
Phase 17 - Reporting	6 h	CSV, JSON, HTML and chart reports.
Phase 18 - Strategy Comparison	6 h	Compare multiple strategies on identical datasets using common performance metrics.
Phase 19 - Optimization	8 h	Profile first, then optimize genuine bottlenecks with NumPy, Numba, joblib or multiprocessing.
Phase 20 - Dashboard	10 h	Optional Streamlit/Plotly interface for selection, execution, trade exploration and comparison.

28. Milestones and Solo Timeline

Milestone	Effort	Meaning
Milestone 1 - Plugin Framework	35-40 cumulative hours	BaseStrategy, context, signals, registry, auto-discovery, CLI and core domain models.
Milestone 2 - Generic Working Backtester	80-90 cumulative hours	Data, strategies, signals, risk, orders, execution, portfolio, trades and analytics all operate end-to-end.
Milestone 3 - ICT Strategy	100-110 cumulative hours	Current ICT logic works inside the reusable framework.
Milestone 4 - Proof of Multi-Strategy Design	110-115 cumulative hours	EMA crossover added without rewriting the engine.

Milestone 5 - Research Platform	140-150 cumulative hours	Database, versioning, comparison, experiments, optimization, dashboard and advanced reporting.
---------------------------------	--------------------------	--

At approximately one focused hour per day, the rough calendar expectation is: plugin framework in about 5-6 weeks, generic functional engine in about 3 months, ICT plus generic engine in about 3.5-4 months, and the fuller research platform in roughly 5 months. Weekend work or faster learning can shorten the calendar duration.

29. What Adding a Future Strategy Should Look Like

Six months later, suppose you invent a London Breakout strategy. The goal is that you create only a strategy package and optionally add any genuinely reusable indicator it needs.

```
strategies/
└-- london_breakout/
    |-- __init__.py
    |-- strategy.py
    |-- config.py
    └-- README.md

class LondonBreakoutStrategy(BaseStrategy):
    name = "London Breakout"
    strategy_id = "london_breakout"
    required_timeframes = ["1m", "15m"]

    def initialize(self, context):
        ...

    def on_candle(self, context):
        ...

    def generate_signal(self, context):
        ...

    def reset(self):
        ...

AVAILABLE STRATEGIES

1. ICT MSS + OTE
2. EMA Crossover
3. RSI Reversion
4. London Breakout
```

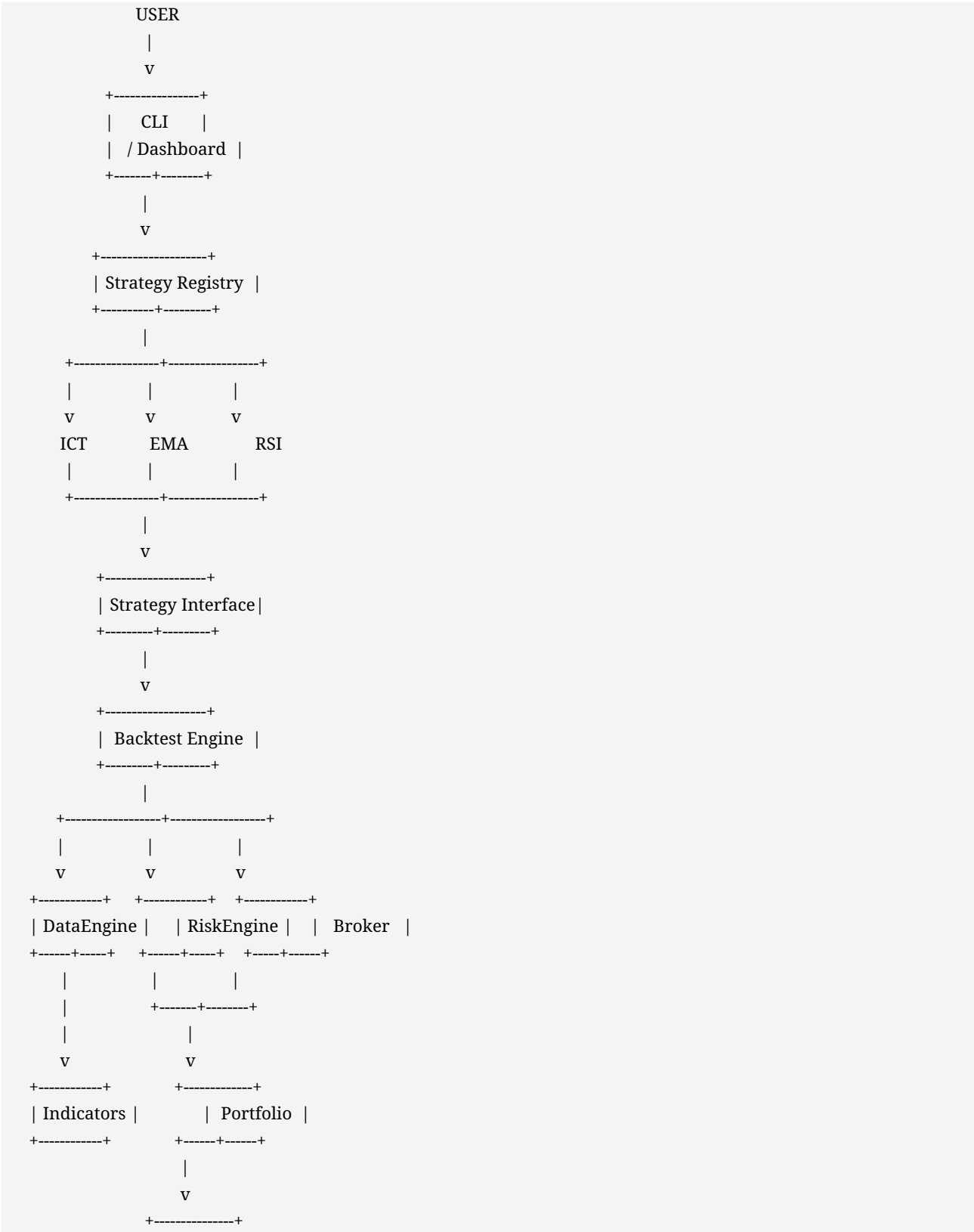
The same DataEngine, IndicatorEngine, BacktestEngine, RiskManager, BrokerSimulator, Portfolio, Analytics, Database and Reporter should handle the new strategy without modification.

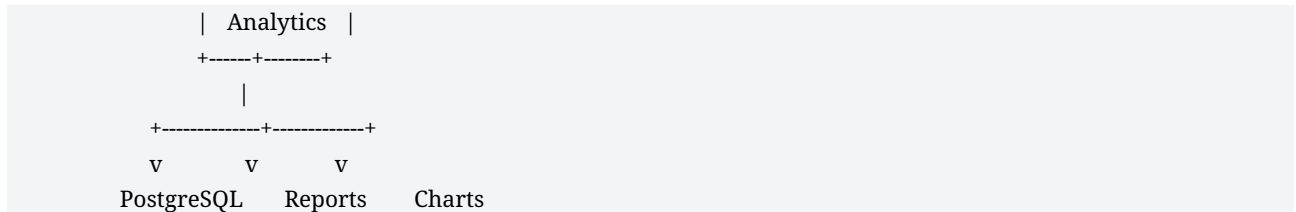
30. Architectural Acceptance Test

The second strategy is the best proof that the architecture succeeded. If implementing an EMA crossover requires modifying BacktestEngine, Broker, Portfolio, RiskManager and DataEngine, the abstractions are

too tightly coupled. Ideally, the second strategy adds only its strategy/configuration package and a reusable EMA indicator if one does not already exist.

31. Long-Term Target Architecture





Core architectural principle: The engine knows how to backtest. The strategy knows when to trade. Neither should know how the other works internally.